

DiagEx: Zero-Shot P&ID-to-DEXPI Digitization with a Vision-Language Agent

Heiko Koziolk, Thilo Braun, Tabea Bordis
ABB Corporate Research Center, Mannheim, Germany
firstname.lastname@de.abb.com

Abstract—Piping and instrumentation diagrams (P&IDs) inform the engineering processes of every process plant, yet the diagrams an automation contractor receives are typically scans or PDFs drawn against customer-specific symbol libraries. Classical computer-vision pipelines reach above 95% symbol detection on in-distribution drawings but require a labeled training corpus per customer, the bottleneck we observe in industrial deployment. We present DiagEx, a pure vision-language-model approach that does not require model training: a single foundation model (Claude Opus 4.7) drives a hand-written ReAct loop over a tile-on-demand viewing layer, emits structured annotations against a DEXPI-aligned schema, and feeds a deterministic reconciliation stage that projects to a pyDEXPI graph. On a public ten-fixture corpus spanning six process domains and four authoring pipelines, a natural-language query engine reaches macro 0.99 accuracy at a median cost of USD 0.07 per query, and the end-to-end P&ID extractor reaches macro F1 0.92 on equipment and 0.90 on instruments and validates against the pyDEXPI schema on every output, but recovers only macro 0.29 edge F1 - a strong inventory and a partial connectivity graph. A GPT-4.1 single-shot baseline, run without tiling, tools, or reconciliation, lands at macro F1 0.57, isolating the contribution of the harness around the model rather than that of the model itself.

Index Terms—piping and instrumentation diagrams, P&ID digitization, vision-language models, ReAct agents, DEXPI, pyDEXPI.

I. INTRODUCTION

Piping and instrumentation diagrams (P&IDs) are the contract between the process and every downstream engineering discipline; control logic, process graphics, hazard reviews, plant simulation, and maintenance handovers all read from them. In a brownfield setting the P&ID an automation contractor receives is, more often than not, a scan or an exported PDF drawn against a customer-specific symbol library, sometimes decades old. Reconstructing a structured graph of equipment, instruments, valves, lines, tags, and connectors from such drawings is the digitization problem.

Two lines of literature tackle the problem from opposite sides and meet only awkwardly in the middle. Classical and deep-learning digitization pipelines [1]–[6] cascade trained symbol detectors, OCR, and line tracers, achieving raw F1 above 95% on the drawings their training corpus represents but degrading sharply on heterogeneous customer conventions, the regime that actually meets a deployment engineer. Language-model-based work [7]–[12] sits *downstream* of digitization, treating the diagram as an already-structured graph or as a single monolithic prompt, and is therefore not a substitute for the front-end recovery problem. Frontier vision-language

models with native high-resolution support, paired with agentic scaffolds such as ReAct [13], now remove the practical obstacle that dominated earlier zero-shot evaluations on dense schematics.

We present DiagEx (Diagram Extractor), a pure vision-language-model pipeline that exploits this combination. A single foundation model manages perception; deterministic Python manages bookkeeping, deduplication, and projection to the target schema. The agent operates against a tile-on-demand viewing layer, never receives the full grid as context, and emits structured output exclusively through a schema-constrained `annotate` tool whose enumerations map cleanly onto pyDEXPI [14], the reference implementation of DEXPI, the open exchange standard for P&IDs. A deterministic reconciliation stage stitches tile-local annotations into a global graph and an optional small-model arbitration pass settles the conflicts that deterministic logic refuses to resolve. The contribution is fourfold: (i) a reusable, diagram-type-agnostic tiling and reconciliation layer; (ii) a ReAct-style agent that autonomously selects among overview, tile, and free-region views; (iii) a deterministic reconciliation stage that preserves an audit trail of low-confidence readings; and (iv) a two-phase deployment in which user-driven queries probe model capability before end-to-end extraction is attempted.

II. RELATED WORK

A. Classical and deep-learning digitization of engineering drawings

Automated digitization of P&IDs has been studied for over two decades, with early hand-crafted approaches reviewed by Moreno-García et al. [15]. Modern work is dominated by multi-stage deep-learning pipelines that decompose the problem into symbol and text detection, line extraction, and topological reconstruction [1]–[5], [16]. Adjacent work by Theisen et al. [6] targets process flow diagrams using a curated corpus of more than 1000 flowsheets. These pipelines achieve raw symbol and text F1 above 95% on in-distribution drawings and establish the current upper bound on detection accuracy; their principal cost is labelled data, as training sets are typically built from drawings of a single company or standard and reported accuracies degrade when transferred to drawings with different symbol conventions or scan qualities [17].

B. Language-model-based approaches to P&IDs and control logic

A more recent thread applies large language models (LLMs) directly to process-engineering artifacts. Koziol et al. [18] prompt GPT-4V with P&IDs to generate IEC 61131-3 ST control code and extend the method with retrieval augmentation and specification-driven synthesis [7], [8]. Alimin et al. [9] take the complementary route of assuming a pre-existing DEXPI model [14], converting it to a labeled property graph and applying graph-based retrieval augmentation for natural-language querying; they note that a single modest P&ID page already expands to roughly 67,000 tokens, which strains context windows. Related work on autocorrection [10], [19] and on generative transformers over SFILES representations of flowsheets [11], [12] similarly operates *downstream* of digitization, leaving the front-end recovery of structured data from raw bitmaps to the classical deep-learning digitization pipelines.

C. Vision-language models and agentic reasoning over images

Foundation vision-language models (VLMs) such as GPT-4V [20] have been evaluated zero-shot on a range of visual reasoning tasks [21], [22], with monolithic single-shot prompting shown to be brittle on dense, high-resolution images. Two mitigations are relevant here: visual prompting to direct model attention [23], and *agentic* scaffolding in which the VLM is placed inside a tool-using loop. The ReAct paradigm of Yao et al. [13] interleaves reasoning with discrete actions so that a model can amortize a large visual workload across many tool calls over bounded views; this pattern has been instantiated for robotics [24] but, to our knowledge, not for engineering drawings. Recent frontier models also offer native high-resolution support (Claude Opus 4.7 accepts up to 3.75 MP per image [25]), removing a practical obstacle that dominated earlier VLM evaluations on dense schematics.

D. Positioning and novelty of this work

DiagEx is positioned as a pure vision-language-model approach distinct from both prior families. In contrast to the deep-learning digitization stacks [1]–[6], we train no symbol, text, or line detector and therefore require no labeled training corpus in the customer’s drawing style, deliberately trading peak in-distribution accuracy for broad zero-shot generalization across heterogeneous customer conventions. In contrast to the LLM-over-graph work [7]–[9], [11], [12], [18], [19], which assumes a digital representation or treats the drawing as a monolithic prompt, we target front-end extraction itself and produce a DEXPI-compliant graph via pyDEXPI [14] directly from bitmap PDF. A hybrid in which a trained detector grounds the agent’s tile choices is a natural evolution our architecture admits without redesign, where labeled data exist and a single drawing convention dominates.

III. APPROACH: DIAGEX METHOD

DiagEx extracts a structured representation of an industrial drawing from a bitmap PDF without any classifier, line detector, or OCR engine specialized to engineering symbols. Five

stages compose the pipeline, illustrated in Figure 1: rasterize and tile the page, present views on demand, annotate under a ReAct loop [13], reconcile, and project to a pyDEXPI [14] graph. Two extractors share these stages, a natural-language query engine and an end-to-end P&ID extractor; only the structured-output schema and an optional arbitration pass distinguish them.

Stage 1, rasterize and tile. DiagEx renders each PDF page through PyMuPDF at a target of 300 DPI, capping the longer side of the bitmap at 6000px so that scanned and vector drawings share a common upper bound. An aspect-aware grid then partitions the page into tiles. Each tile holds a budget of 2200 image tokens, roughly 1.65 megapixels at the Opus 4.7 vision token rate of one token per 750 pixels [25], and shares 20% overlap with its neighbors on each axis so that any symbol straddling a tile boundary appears intact in at least one neighbor. Stable identifiers of the form `p{page}-r{row}-c{col}` survive across reruns and link transcripts, cost logs, and reconciled annotations to a specific patch of pixels.

Stage 2, view on demand. The agent never sees the full tile grid as context. It first calls `get_overview` to obtain a downsampled view of the whole page (capped at 2000px on the longer side), then chooses which tiles or arbitrary regions to inspect through `list_tiles`, `get_tile`, and `get_region`. This pattern bounds the visual workload of any one model call and lets the agent amortize its decision-making across many short calls instead of one monolithic prompt. The view provider records an affine transform with each view; the runtime later projects locally emitted coordinates back to the page frame, sparing the model from reasoning in a multi-megapixel global coordinate space and avoiding a common class of off-by-tile hallucinations.

Stage 3, agentic annotation. Claude Opus 4.7 [25] drives the loop through a hand-written ReAct [13] runtime of roughly 350 lines of Python. We deliberately avoid an orchestration framework so that prompt construction, caching, and cost accounting remain auditable. The agent emits structured output exclusively through the `annotate` tool, whose schema enumerates six annotation kinds (equipment, valve, instrument, line, tag, off-page connector) and exposes the DEXPI attributes the downstream builder consumes (`equipment_class`, `valve_type`, `instrument_function`, `measured_variable`, `loop_number`, `direction`). A single class registry generates this schema and the builder’s lookup tables, so the agent and the deterministic stages share one source of truth. `list_annotations` lets the agent inspect what it has already recorded; `finish` ends the loop. The P&ID extractor adds a `lookup_symbol` tool that resolves labels against an extracted legend when that legend overflows the prompt budget. Four effort profiles (low, medium, high, xhigh) trade reasoning depth for step budget at six, twenty, forty, and sixty steps; the query engine defaults to `high`, the P&ID extractor to `medium`. Prompt caching covers the system prompt, the rendered DEXPI ontology, the tool schemas, and

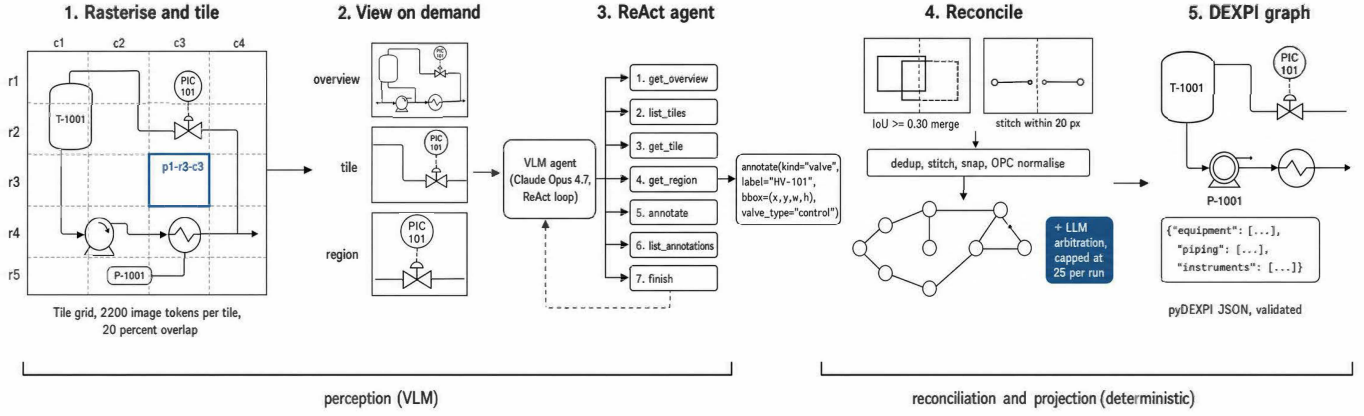


Fig. 1. The DiagEx pipeline. Perception (Stages 1 to 3) is performed by a single vision-language model in a tool-using loop; reconciliation and DEXPI projection (Stages 4 and 5) are deterministic Python. The agent never receives the full tile grid as context; it requests overview, tile, and region views on demand.

the overview image, which together dominate the repeated input across tile calls.

Stage 4, reconcile. When the agent emits *finish* for a page, deterministic Python folds its annotations into a single graph. Two symbols merge when their bounding boxes show an intersection-over-union of at least 0.30 and their normalized labels agree; the runtime keeps the highest-confidence reading and stores the rest in an *alternate_readings* field. Overlaps in the gray-zone window from 0.15 to 0.30 surface as conflicts rather than silently collapse. Line endpoints within 20 px of one another across a tile boundary stitch together, and graph edges snap their endpoints to the nearest node within 30 px. Off-page connectors match across sheets by normalized label equality (uppercase, alphanumeric only, leading zeros collapsed), the only cross-sheet operation DiagEx performs deterministically. The P&ID extractor adds a tool-free arbitration pass aimed only at the conflicts deterministic logic refuses to settle, namely gray-zone duplicates, OCR flip candidates, and ambiguous off-page connector clusters of three or more. The runtime sends each conflict, cropped with roughly 40 px of padding, to a single small model call with no further tool access and a tight output budget. A hard cap of twenty-five arbitrations per run bounds the worst-case spend, and one record per conflict lands in *arbitration.jsonl* for audit. On the canonical DEXPI reference fixture this pass cut the run-to-run standard deviation of entity counts from 29.3 % to 10.6 % of the mean at a corpus-wide marginal cost below one US dollar.

Stage 5, project to DEXPI. A deterministic builder maps the reconciled graph to a pyDEXPI [14] model through the same class registry the *annotate* tool exposes. Classes that pyDEXPI 1.2 omits (notably *Reactor* and the three-way valve) degrade to *CustomEquipment* or *CustomOperatedValve*, with the original registry key preserved as *typeName* so that future pyDEXPI releases can re-resolve them without re-extraction. The query engine reuses Stages 1 to 4 unchanged and replaces Stage 5 with

TABLE I
PUBLIC P&ID CORPUS USED IN THE EVALUATION. ALL TEN FIXTURES ARE SINGLE-PAGE DRAWINGS DECLARED IN EVAL/DATASETS/ETFA2026.MANIFEST.YAML.

Stem	Type	Domain	Annotation
dexpi-reference	vector	DEXPI synthetic	full + ref. XML
two-tanks	raster	process utility	partial
butane1	raster	C4 cracking	full
butane2	raster	C4 cracking, alt	partial
tennessee1	vector	Tennessee Eastman	full
grit-washer1	raster	wastewater grit	partial
open100-1	raster	PWR, CVCS	partial
open100-2	raster	PWR, RHR	partial
open100-3	raster	PWR, RCS	partial
open100-4	raster	PWR, pressurizer	partial

TABLE II
PHASE I QUERY FAMILIES. EACH FIXTURE CARRIES ONE QUERY PER FAMILY, FORTY QUERIES IN TOTAL.

Family	Template (filled per diagram)	Scoring
Inventory	List every control valve and tag.	set match
Counting	How many LT/LIT instruments?	numeric $\pm 5\%$
Connectivity	Path from tag <i>A</i> to tag <i>B</i> ?	exact (yes/no)
Identification	Equipment at page coords (<i>x</i> , <i>y</i>)?	contains

a small answerer that derives counts, set memberships, and connectivity verdicts from the same reconciled graph.

The two extractors play complementary roles in the evaluation in Section V. The query engine probes model capability on heterogeneous drawings without requiring full DEXPI ground truth; the P&ID extractor produces a complete pyDEXPI graph end to end. Every run writes a full transcript, a per-step token and cost breakdown, the reconciled graph, and (for the P&ID extractor) the validated pyDEXPI JSON to *runs/<id>/*. These artifacts ground both reproducibility and the cost analysis we report below.

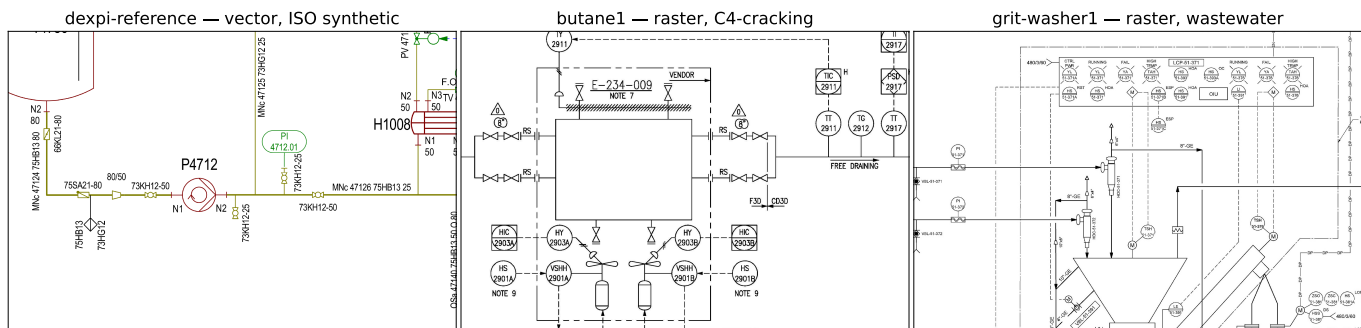


Fig. 2. Three representative crops from the corpus, illustrating the visual heterogeneity DiagEx must handle without per-customer training. Left: *dexpi-reference*, the synthetic ISO-10628 reference drawing rendered from a vector source. Centre: *butane1*, a dense C4-cracking sheet exported as a high-resolution raster through Adobe Acrobat 26.1, with a different symbol convention and tag-block style. Right: *grit-washer1*, a third-party engineered wastewater drawing in raster form, with yet another symbol library and a much denser instrument-and-control mix on a small page. The three fixtures span three domains, three authoring pipelines, and the corpus’s vector-raster split.

IV. EVALUATION SETUP

Corpus. The evaluation runs on ten public single-page P&IDs listed in Table I. We assembled the set to span the kinds of drawings a process engineer meets in practice: a synthetic DEXPI reference drawing, a heavily instrumented two-tank utility sheet, a C4-cracking train drawn twice through two different export pipelines, the Tennessee-Eastman flowsheet redrawn from public sources, a wastewater grit washer, and four sheets of an open-source small modular reactor (the chemical and volume control, residual heat removal, reactor coolant, and pressurizer-auxiliary sheets). Together the corpus covers six process domains and four authoring or export pipelines, with eight raster-based and two vector-based sheets; Figure 2 shows three representative crops that illustrate the resulting visual variety. Two within-corpus clusters, the two butane drawings and the four reactor sheets, isolate *rendering* variance from *process* variance: in each cluster the underlying process is the same and only the rendering pipeline or the sheet differs. We deliberately excluded the PDFs that carried the strongest pre-training memorization risk; the residual risk on *tennesseel* is acknowledged in Section VI.

Ground-truth authoring. Building a 100-entity DEXPI graph by hand is impractical: every DEXPI-conformant drawing tool ships inside a commercial CAE suite, and pyDEXPI [14] is a Python library with no drawing canvas. We therefore adopt a deliberate *bootstrap-then-correct* workflow. DiagEx produces a first-pass extraction at its highest reasoning budget, then a human rater steps through every candidate entity in a custom annotation tool and confirms, revises, rejects, or adds. For three fixtures (*dexpi-reference*, *butane1*, *tennesseel*) we author full DEXPI truth, both the entity inventory and the connectivity graph; for the remaining seven we author entity-inventory truth only, since edges are far more expensive to author and the inventory alone supports per-kind precision and recall. We grade *dexpi-reference* against the canonical DEXPI C01 reference XML rather than against our own bootstrap, providing an external anchor independent of any rater judgment. Single-rater authoring is a threat to

validity discussed in Section VI.

Phase 1 metrics. Each fixture carries one query per family in Table II, for forty queries in total. Each scoring kind matches the structure of the answer: *set match* compares unordered tag sets up to a normalized string equality (so “FIC-101” and “FIC 101” agree); *numeric $\pm 5\%$* accepts a count within five percent of the rater’s count; *exact* requires a yes-or-no answer to match; *contains* requires the predicted equipment-class label to be a substring of the rater’s label, so “centrifugal pump” is accepted against a rater label of “pump”. The Connectivity family enforces a five-yes / five-no contract across the corpus, so a model that always answers “yes” cannot score 50%; a typical filled instance reads “Is there a process flow path from V-101 to E-205?”. We authored Identification coordinates in Acrobat against the rendered PDF rather than deriving them from the system’s own output. We report per-fixture accuracy, the macro mean across the ten fixtures, and median cost and latency per query; targets restated from the specification are macro inventory ≥ 0.90 , macro connectivity ≥ 0.80 , median cost below USD 0.50 per query, and median latency below two minutes. Vision-language models are not bit-deterministic even at zero temperature, so we run each query three times and report the mean and standard deviation.

Phase 2 metrics. For each fixture we compute per-kind node F1 (the harmonic mean of precision and recall) for the three node kinds equipment, valve, and instrument; tag exact match (EM) on the optical character recognition (OCR) of every tag string; edge F1 on the three full-truth fixtures; and the pyDEXPI schema-validation pass rate on all ten outputs. A predicted node matches a ground-truth node when, sharing the same kind, they pass any of four signals: their bounding boxes overlap by at least an intersection-over-union (IoU) of 0.30, their normalized labels agree, they share a normalized tag-root extracted from a verbose label, or (for off-page connectors (OPCs) only) their first tokens overlap. The label-side signals catch verbose model labels like XV-2151 (shutdown valve, left feed-gas inlet) matching the bare-tag truth XV-2151, and protect against incidental string differences between the two pipelines (a stray space, an upper-

case versus lower-case kind name). Tag EM is computed on the same matched pairs and is then strict on the label itself: given the entity is identified, the predicted tag must equal the truth after normalization, so a prediction of FIC-IO2 against a truth of FIC-102 counts as a miss, since downstream consumers of the DEXPI graph (control-logic derivation, tag look-up) key on the tag string and treat tags as unique identifiers. We deliberately do not penalize correctly-OCR'd tags whose bounding box happens to fall below the IoU threshold, because the downstream consumer cares about the tag, not the pixel coordinates. We report cost in US dollars per sheet and report latency in input, output, and image tokens, the figure a reader can reproduce, with wall-clock seconds as a secondary column. Targets restated from the specification are equipment F1 ≥ 0.85 , instrument F1 ≥ 0.80 , tag EM ≥ 0.85 , edge F1 ≥ 0.70 , and validation pass rate ≥ 0.50 .

Conditions. The evaluation runs three conditions: baseline, no-tile, and GPT4.1 single-shot. The *baseline* matches the configuration described in Section III. A *no-tile* ablation disables the Stage 1 grid and feeds the entire page as a single image, downsampled only as far as the model's largest supported image; it runs on the three full-truth fixtures and isolates the contribution of tiling from the contribution of the underlying model, answering the obvious reviewer question of whether a frontier vision-language model could simply consume the whole page in one shot. A *GPT-4.1 single-shot* baseline runs on all ten fixtures with one chat-completions call per Phase 1 query — no tool-use loop, no tile grid, and no reconciliation — anchoring how much of the result depends on the harness around the model rather than on the model itself; the cross-model leg is Phase 1 only, since the Anthropic and OpenAI tool-use schemas differ in four load-bearing details (image-block shape, tool-result shape, image-inside-tool-result support, and the cache-control marker that underpins our cost story) and porting the harness across SDK families is out of scope at this page budget. We deliberately scoped out the full ablation grid (overlap, effort, legend caching, prompt caching) at this page budget; the single ablation result and an error-class breakdown by failure mode are reported inline in Section V.

Harness. A single command runs every condition–fixture pair, writes per-run artifacts (transcript, per-step token and cost log, reconciled graph, P&ID extractor output) into a structured output tree, and emits the canonical results table from which every table and figure in Section V is generated as a LaTeX fragment the manuscript includes directly. The supplementary package ships HTTP-level cassettes that replay every Phase 1 query deterministically without an Anthropic API key and pins the model snapshot identifier.

V. RESULTS

We report Phase 1 query accuracy and Phase 2 extraction quality on the ten-fixture corpus of Section IV, then isolate the contribution of tiling and anchor the result against a frontier non-Anthropic VLM.

TABLE III
PHASE 1 QUERY ACCURACY PER DIAGRAM AND QUERY FAMILY. CELLS ARE F1 (SET_MATCH), 0/1 (OTHER SCORING KINDS) AVERAGED ACROSS $N = 3$ TRIALS. COST / LATENCY ARE PER-QUERY MEDIANS. THE MACRO ROW WEIGHTS ALL TEN FIXTURES EQUALLY.

Diagram	Inv.	Cnt.	Conn.	Ident.	Overall	Cost	Lat.
butane1	1.00	1.00	1.00	1.00	1.00	\$ 0.06	23.4
butane2	1.00	1.00	0.67	1.00	0.92	\$ 0.18	47.5
dexpi-reference	1.00	1.00	1.00	1.00	1.00	\$ 0.03	11.2
grit-washer1	1.00	1.00	1.00	1.00	1.00	\$ 0.07	22.2
open100-1	0.97	1.00	1.00	1.00	0.99	\$ 0.08	24.9
open100-2	1.00	1.00	1.00	1.00	1.00	\$ 0.05	16.6
open100-3	1.00	1.00	1.00	1.00	1.00	\$ 0.08	24.5
open100-4	1.00	1.00	1.00	1.00	1.00	\$ 0.11	31.3
tennessee1	1.00	1.00	1.00	1.00	1.00	\$ 0.03	12.3
two-tanks	1.00	1.00	1.00	1.00	1.00	\$ 0.07	22.9
macro	1.00	1.00	0.97	1.00	0.99	\$ 0.07	23.2

A. Phase 1: query engine

Across forty queries and three trials per query, the macro overall accuracy is 0.99 (Table III); all four exit criteria from Section IV clear by margins of factors rather than percentages. Eight of ten fixtures score 1.00 overall; the two sub-perfect fixtures are *butane2* at 0.92, where a single connectivity trial out of three returns the wrong yes-or-no answer, and *open100-1* at 0.98, where a single inventory trial drops one tag from a five-tag truth set ($F1 = 0.91$ on the inventory family).

The tool-call mix explains the cost: across 474 ReAct steps the agent issued 440 tool calls, of which $\approx 69\%$ were `get_region` and $\approx 28\%$ `get_overview`; the regular tile grid is rarely needed for query-shaped questions. Prompt caching serves 59% of input tokens at the discounted cache-read rate, the mechanism behind the seven-fold cost margin.

B. Phase 2: P&ID extraction

Macro per-kind F1 is 0.92 for equipment and 0.90 for instruments across all ten fixtures, and the pyDEXPI schema validator accepts all ten outputs (Table IV). On the three full-truth fixtures, OPC F1 macro is 0.87 and edge F1 macro is 0.29. Five fixtures reach equipment F1 1.00 (*butane1*, *butane2*, *open100-1*, *open100-2*, *open100-3*); the floor is *grit-washer1* at 0.67, where the agent over-generates LCP-panel and gate-side switch entities that the rater excluded. Instrument F1 ranges from 0.79 on *open100-3* to 1.00 on four fixtures.

Tag EM macro is 0.83, two points shy of the 0.85 floor; the per-fixture range runs from 0.67 on *dexpi-reference* to 1.00 on *open100-2*, eight of ten fixtures land at or above 0.72. Across the corpus the residual budget is 103 of 563 truth tags: 65 truth nodes (11.5%) are missed entirely (no same-kind, IoU-or-label-eligible prediction at all) and 38 of the 498 matched pairs (7.6%) recover the correct symbol with the correct kind but a tag string that differs from the truth after normalization, the residual OCR confusion that a strict character-equal contract drops.

TABLE IV

PHASE 2 DEXPI EXTRACTION PER DIAGRAM. OPC F1 AND EDGE F1 ARE N/A FOR INVENTORY-ONLY FIXTURES (NO OPCs / EDGES LABELLED IN GROUND TRUTH); Y IN THE DEXPI COLUMN MEANS THE PYDEXPI SCHEMA VALIDATOR ACCEPTS THE OUTPUT. THE MACRO ROW WEIGHTS ALL TEN FIXTURES EQUALLY FOR THE PER-KIND F1 COLUMNS AND ONLY THE THREE FULL-TRUTH FIXTURES FOR OPC F1 AND EDGE F1.

Diagram	Entities	Equipment F1	Instrument F1	OPC F1	Tag EM	Edge F1	DEXPI	Cost	Minutes
butane1	54	1.00	1.00	0.80	0.76	0.21	Y	\$ 6.59	13.8
butane2	55	1.00	0.85	—	0.87	—	Y	\$ 0.63	2.4
dexpi-reference	18	0.94	1.00	0.91	0.67	0.40	Y	\$ 2.99	8.9
grit-washer1	57	0.67	0.86	—	0.89	—	Y	\$ 6.34	13.5
open100-1	50	1.00	0.85	—	0.94	—	Y	\$ 1.44	2.9
open100-2	26	1.00	1.00	—	1.00	—	Y	\$ 1.00	2.2
open100-3	44	1.00	0.79	—	0.80	—	Y	\$ 2.10	9.0
open100-4	55	0.91	0.85	—	0.75	—	Y	\$ 5.54	22.3
tennessee1	70	0.90	1.00	0.91	0.89	0.25	Y	\$ 1.20	3.6
two-tanks	134	0.78	0.82	—	0.72	—	Y	\$ 10.35	11.4
macro	563	0.92	0.90	0.87	0.83	0.29	Y	\$ 38.17	90.0

cost vs. complexity, by source type and F1

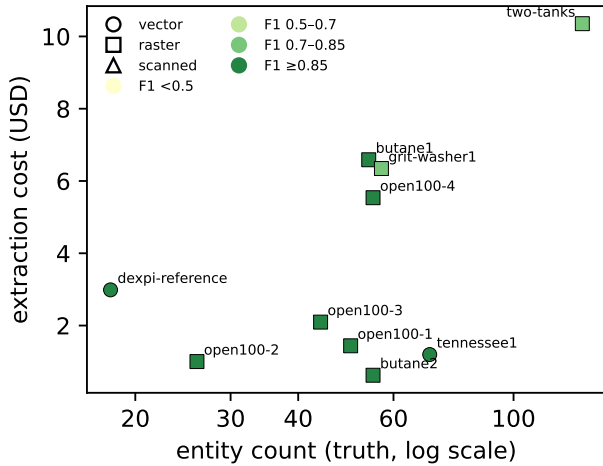


Fig. 3. Phase 2 extraction cost in US dollars against the ground-truth entity count of each fixture (log-scale x). Marker shape encodes source type, marker color the overall-F1 bucket. Cost grows roughly linearly with complexity; the dense raster fixtures *two-tanks* and *butane1* sit above the trend, the small vector fixtures *tennessee1* and *dexpi-reference* below.

Connectivity is the principal gap. On the three full-truth fixtures, edge F1 macro is 0.29 against the 0.70 floor — a factor-of-two-plus shortfall — and DiagEx as it stands recovers a strong entity inventory but only a partial connectivity graph. The dominant failure mode is unpacked in Section V-E.

Figure 3 plots extraction cost against ground-truth entity count, with marker shape encoding source type and color the overall-F1 bucket. The dense raster fixtures *two-tanks* (USD 10.35 at 134 truth entities) and *butane1* (USD 6.59 at 54 truth entities) sit above the trend; the small vector fixtures *tennessee1* (USD 1.20, 70 entities) and *dexpi-reference* (USD 2.99, 18 entities) sit below. Aggregated end-to-end reproduction cost across all four legs is USD 68.01 over $\approx 3\text{h}21\text{min}$ of wall-clock time; the supplementary package ships every transcript, per-step cost log, reconciled graph, validated pyDEXPI JSON, and HTTP-

level cassettes that replay Phase 1 without an Anthropic API key.

C. Tiling ablation

The single ablation removes the Stage 1 grid and feeds the entire page to Opus 4.7 at the model’s 3.75 MP image cap. Across the three full-truth fixtures, mean equipment F1 falls from 0.95 to 0.76 (delta -0.19), mean instrument F1 from 1.00 to 0.74 (delta -0.26), and mean tag EM from 0.77 to 0.60 (delta -0.17); mean edge F1 moves from 0.29 to 0.33 (delta $+0.05$). DEXPI validation passes on all six runs. The bottleneck is visual resolution rather than the model: per-kind F1 takes the largest hit, and *butane1* (dense raster, large page) collapses end-to-end, equipment F1 $1.00 \rightarrow 0.56$, instrument F1 $1.00 \rightarrow 0.37$, tag EM $0.76 \rightarrow 0.30$. Cost rises rather than falls when tiling is removed, mean cost per fixture USD $3.79 \rightarrow 5.12$ (+35%) and wall-clock per fixture $536 \rightarrow 846\text{s}$, because the agent compensates with many more `get_region` zoom-ins (*butane1* ablation: 139 steps against 37 baseline). Tiling pays for itself.

D. Cross-model anchor

The GPT-4.1 single-shot leg of Section IV runs all forty Phase 1 queries through a single chat-completions call per query against the same fixture set; total cost is USD 0.11 over three minutes of wall-clock time. The macro overall accuracy is 0.57, against 0.99 for the baseline; per family, the leg holds connectivity at 0.90 but loses on counting (0.30), inventory (0.48), and identification (0.60). The reading is not “same harness, different model”; the four schema deltas listed in Section IV prevent a like-for-like comparison at this page budget. The leg therefore anchors what a frontier non-Anthropic VLM does given just the page and the question, before the harness adds anything; like the tiling ablation, this isolates the contribution of the harness rather than the model.

E. Failure modes

The residual error budget is dominated by four classes. *Edge segmentation miss*: 47 truth edges unrecovered across the three full-truth fixtures (8/10 *butane1*, 10/16

dexpi-reference, 29/36 tennessee1); this pins the 0.29 macro edge F1 and is unpacked in §VI-A. *Inventory recall miss*: 65 Phase 2 truth nodes unmatched across all ten fixtures (23 equipment, 39 instrument, 3 OPC), with the largest single contribution from two-tanks; the same class fires only once across 120 Phase 1 trials. *Tag OCR near-miss*: 38 of 498 Phase 2 matched pairs (7.6%) recover the correct kind but a tag string that differs after normalization (typical confusions: O/0, S/5, leading-zero collapse, verbose suffixes such as TI-2916 H/L vs. TI-2916). *Connectivity flip*: one Phase 1 trial on butane2-q3 returns the opposite yes-or-no answer to the other two trials, the only non-determinism that survives temperature=0.0. The residual concentrates in tag-glyph confusion and segment-at-valve omissions rather than wholesale symbol misclassification.

VI. DISCUSSION AND LIMITATIONS

A. What the numbers mean

The headline that a single foundation model with a hand-written ReAct loop reaches macro 0.99 on Phase 1 and macro $F1 \geq 0.90$ on Phase 2 equipment and instrument detection without a single labeled symbol example argues, against the deep-learning digitization literature [1], [3]–[5], that for query-shaped questions the bottleneck has moved from *detection* to *interface*. The same does not hold for connectivity: edge F1 of 0.29 against a 0.70 floor is a factor-of-two-plus shortfall, and DiagEx as it stands produces a strong entity inventory but only a partial connectivity graph. The dominant failure (single polylines traced through inline valves rather than split into the three DEXPI segments the schema expects) is a segmentation problem prompting and reconciliation may bound but have not yet solved at the precision and recall downstream consumers — control-logic derivation, hazard review, simulation — need; closing it is unfinished work, not a near-miss. Together, the tiling ablation and the GPT-4.1 leg are bookends on a single conclusion: the harness — tiling, the ReAct loop, and deterministic reconciliation — drives most of the result. The cross-model leg cannot establish model parity, since GPT-4.1 runs without the harness our SDK port does not yet support, and we make no such claim.

B. Practical implications

The pure-VLM approach is attractive beyond the F1 numbers. Against per-customer trained CV pipelines [1], [3]–[5], DiagEx requires no labeled symbol corpus and no per-vendor retraining, both of which dominate deployment time and labor cost in practice; the eight-to-two raster bias and four authoring pipelines our corpus already covers are precisely the heterogeneity those stacks degrade on. Against the alternative of hand-authoring DEXPI in a commercial CAE suite, the average extraction cost of USD 3.82 per sheet is roughly two orders of magnitude below typical loaded labor rates and compounds across hundreds of pages in a brownfield handover. The bootstrap-then-correct workflow is itself the realistic deployment pattern — an agent first-pass that a domain expert reviews — though the residual tag-OCR and

edge-F1 gaps mean fully unattended extraction is not yet appropriate for safety-critical use cases.

C. Threats to validity

Internal validity. The Phase 2 ground truth is bootstrap-then-correct from DiagEx’s own first-pass, which risks self-grading. The audit logs of the truth-authoring sessions quantify how much the rater diverged from the bootstrap on the three full-truth fixtures: of 155 model-proposed nodes, 32.9% (51) were rejected outright, 60 to 78% of the retained nodes had their bbox or label edited, and the rater added 22 nodes the system had not proposed (14.3% of the final 154 ground-truth nodes); for edges, 35.7% of the final 70 ground-truth edges entered through the rater’s *add* action rather than from the bootstrap. The 14.3% node-add rate caps how much self-grading can hide on the inventory side, since entities the system never proposes can only enter via that channel; the high edge-add share confirms the rater was not anchored to the model’s connectivity. We further mitigate by grading dexpi-reference against the canonical DEXPI C01 reference XML, an external anchor independent of any rater judgment, and acknowledge that a second-rater pass remains future work.

Construct validity. Tag EM is computed conditional on the entity being identified — same kind, with either spatial overlap or a label-side signal — and applies a strict character-equal contract to the label itself, so a single OCR confusion drops a row that is otherwise categorically correct; the 7.6% near-miss share is the residual readability error that survives that contract. Per-kind node F1 still uses the $\text{IoU} \geq 0.30$ overlap test as one of its match signals; the threshold is liberal by detection-paper standards, justified by the manual-bbox noise floor we observed during VIA correction. Edge F1 collapses missing and over-segmented edges into one number; a graph-edit-distance metric would separate them and is queued for the journal version.

External validity. Ten fixtures across six process domains is small, and the pre-training contamination risk on tennessee1 is real even after we removed the two published-paper PDFs: the Tennessee-Eastman flowsheet is famously well-documented, so any number on tennessee1 should be read as an upper bound. The corpus is English and ISA-style only; performance on Asian or European symbol standards, multi-page assemblies, and hand-annotated legacy scans is unknown.

Conclusion validity. The Phase 1 ground truth and the three full-truth Phase 2 graphs are single-rater. We did not run an inter-rater κ pass; without one, rater bias on borderline taxonomy decisions and on connectivity calls where the source PDF is ambiguous is not bounded. Three trials per query is enough to surface non-determinism but too few to estimate variance tightly; vision-language models are not bit-deterministic even at temperature=0.0, an observation visible in the single butane2-q3 flip and in the run-to-run entity-count variance the arbitration pass cuts on the canonical reference fixture (Section III).

D. Future work

Three directions are immediate: hybridization, in which a specialized symbol detector grounds the agent’s tile choices; a cross-SDK port that lets Phase 2 run end-to-end on GPT-class models; and reusing the harness for other diagram families such as logic diagrams, where only the schema swaps.

VII. CONCLUSION

DiagEx shows that a frontier vision-language model, wrapped in a small tool-using harness, can digitize P&IDs at a quality and cost that zero-shot methods could not reach a year ago. On the ten-fixture public corpus, the pipeline answers natural-language questions about each drawing almost perfectly and recovers the inventory of equipment and instruments at high precision and recall — with the connectivity graph remaining only partial — without any training corpus or customer-specific symbol library, and at a few US dollars per sheet, well below the labor of hand-authoring the equivalent in a commercial CAE suite.

The lesson is not that vision-language models have solved the diagram on their own. A single-shot run of a frontier non-Anthropic model answers only about half the queries correctly, and turning off tiling alone visibly degrades extraction. What does the work is the harness: tile-on-demand views, an agent that decides where to look, a schema the model is forced to fill rather than invent, and a deterministic reconciliation stage that stitches annotations into a graph. The model is a perception engine; the engineering sits around it. The same harness extends without redesign to logic diagrams, single-line diagrams, and cause-and-effect matrices, suggesting that diagram digitization may finally be on its way to becoming a software problem rather than a data-collection one.

AI TOOL USAGE

During this work the authors used Generative AI to assist in manuscript editing and code implementation. All research methodology, analysis, and conclusions are the authors’ original contributions.

REFERENCES

- [1] R. Rahul, S. Paliwal, M. Sharma, and L. Vig, “Automatic information extraction from piping and instrumentation diagrams,” in *Proceedings of the 8th International Conference on Pattern Recognition Applications and Methods (ICPRAM)*, 2019.
- [2] S. Mani, M. A. Haddad, D. Constantini, W. Douhard, Q. Li, and L. Poirier, “Automatic digitization of engineering diagrams using deep learning and graph search,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2020, pp. 176–177.
- [3] S. Paliwal, A. Jain, M. Sharma, and L. Vig, “Digitize-PID: Automatic digitization of piping and instrumentation diagrams,” in *Trends and Applications in Knowledge Discovery and Data Mining, PAKDD 2021 Workshops*, ser. Lecture Notes in Computer Science, vol. 12705. Springer, 2021, pp. 168–180.
- [4] H. Kim, W. Lee, M. Kim, Y. Moon, T. Lee, M. Cho, and D. Mun, “Deep-learning-based recognition of symbols and texts at an industrially applicable level from images of high-density piping and instrumentation diagrams,” *Expert Systems with Applications*, vol. 183, p. 115337, 2021.
- [5] J.-B. Kim, Y. Moon, S.-T. Han, and D. Mun, “Automated inspection of P&ID object recognition using deep learning,” *Scientific Reports*, vol. 15, 2025.
- [6] M. F. Theisen, K. N. Flores, L. Schulze Balhorn, and A. M. Schweidtmann, “Digitization of chemical process flow diagrams using deep convolutional neural networks,” *Digital Chemical Engineering*, vol. 6, p. 100072, 2023.
- [7] H. Koziolok, S. Grüner, R. Hark, V. Ashiwal, S. Linsbauer, and N. Eskandani, “LLM-based and retrieval-augmented control code generation,” in *Proceedings of the 1st International Workshop on Large Language Models for Code (LLM4Code)*, 2024, pp. 22–29.
- [8] H. Koziolok, T. Braun, V. Ashiwal, S. Linsbauer, M. A. Hansen, and K. Grotterud, “Spec2Control: Automating PLC/DCS control-logic engineering,” in *Proceedings of the 48th International Conference on Software Engineering (ICSE 2026)*, SEIP, Rio de Janeiro, Brazil, 2026.
- [9] A. A. Alimin, D. P. Goldstein, L. Schulze Balhorn, and A. M. Schweidtmann, “Talking like piping and instrumentation diagrams (P&IDs),” in *Proceedings of the 35th European Symposium on Computer Aided Process Engineering (ESCAPE35)*, Ghent, Belgium, 2025.
- [10] L. Schulze Balhorn, M. Caballero, and A. M. Schweidtmann, “Toward autocorrection of chemical process flowsheets using large language models,” in *Computer Aided Chemical Engineering*. Elsevier, 2024, vol. 53, pp. 3109–3114.
- [11] E. Hirtreiter, L. Schulze Balhorn, and A. M. Schweidtmann, “Toward automatic generation of control structures for process flow diagrams with large language models,” *AIChE Journal*, vol. 70, no. 1, p. e18259, 2024.
- [12] G. Vogel, L. Schulze Balhorn, and A. M. Schweidtmann, “Learning from flowsheets: A generative transformer model for autocompletion of flowsheets,” *Computers & Chemical Engineering*, vol. 171, p. 108162, 2023.
- [13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [14] D. P. Goldstein, L. Schulze Balhorn, A. A. Alimin, and A. M. Schweidtmann, “pyDEXPI: A Python framework for piping and instrumentation diagrams using the DEXPI information model,” in *Proceedings of the 35th European Symposium on Computer Aided Process Engineering (ESCAPE35)*, Ghent, Belgium, 2025.
- [15] C. F. Moreno-García, E. Elyan, and C. Jayne, “New trends on digitisation of complex engineering drawings,” *Neural Computing and Applications*, vol. 31, no. 6, pp. 1695–1712, 2019.
- [16] S.-T. Han, Y. Moon, H. Lee, and D. Mun, “Rule-based continuous line classification using shape and positional relationships between objects in piping and instrumentation diagram,” *Expert Systems with Applications*, vol. 248, p. 123366, 2024.
- [17] H. Bhanbhro, Y. K. Hooi, Z. Hassan, and F. Mohamed Nordin, “A review of deep learning methods for digitisation of complex documents and engineering diagrams,” *Artificial Intelligence Review*, vol. 57, no. 200, 2024.
- [18] H. Koziolok and A. Koziolok, “LLM-based control code generation using image recognition,” in *Proceedings of the 1st International Workshop on Large Language Models for Code (LLM4Code)*, 2024.
- [19] L. Schulze Balhorn, N. Seijssener, K. Dao, M. Kim, D. P. Goldstein, G. H. M. Driessen, and A. M. Schweidtmann, “Rule-based autocorrection of piping and instrumentation diagrams (P&IDs) on graphs,” in *Proceedings of the 35th European Symposium on Computer Aided Process Engineering (ESCAPE35)*, Ghent, Belgium, 2025.
- [20] OpenAI, “GPT-4V(ision) system card,” OpenAI, Tech. Rep., 2023. [Online]. Available: <https://openai.com/index/gpt-4v-system-card/>
- [21] W. Wu, H. Yao, M. Zhang, Y. Song, W. Ouyang, and J. Wang, “GPT4Vis: What can GPT-4 do for zero-shot visual recognition?” *arXiv preprint arXiv:2311.15732*, 2024.
- [22] S. Singh, G. Pavlakos, and D. Stamoulis, “Evaluating zero-shot GPT-4V performance on 3D visual question answering benchmarks,” *arXiv preprint arXiv:2405.18831*, 2024.
- [23] K. Singh, M. Khanna, A. Biswas, P. Moturi, and Shivam, “Visual prompting methods for GPT-4V based zero-shot graphic layout design generation,” in *Tiny Papers @ ICLR 2024*, 2024.
- [24] Z. Wang, R. Shen, and B. Stadie, “Solving robotics problems in zero-shot with vision-language models,” *arXiv preprint arXiv:2407.19094*, 2024.
- [25] Anthropic, “Introducing Claude Opus 4.7,” <https://www.anthropic.com/news/claude-opus-4-7>, 2026, accessed: 2026-04-22.